REPORTE TAREA 1
ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Vicente Carvajal Helqui

25 de marzo de 2026

00:17

1. Entorno experimental

Hardware Utilizado:

-CPU: Intel Core i5 1165G7

-RAM: 8GB DDR4 3200MHz

-Almacenamiento: 256GB Nvme

-SO: Linux Debian 13 Trixie

-Compilador: g++17

Software Utilizado:

-g++

-Visual Code Studio

-Python 3

-TexLive

Datasets utilizados:

Carpeta array_input con textos que poseen arreglos desordenados

Carpeta matrix_inputs con textos que poseen las matrices a multiplicar

2. Resultados y Análisis

Estos son los resultados de estos experimentos en cuestion

Empezaremos mostrando los resultados de los algoritmos de ordenamiento de arreglos (Sort, MergeSort y QuickSort)

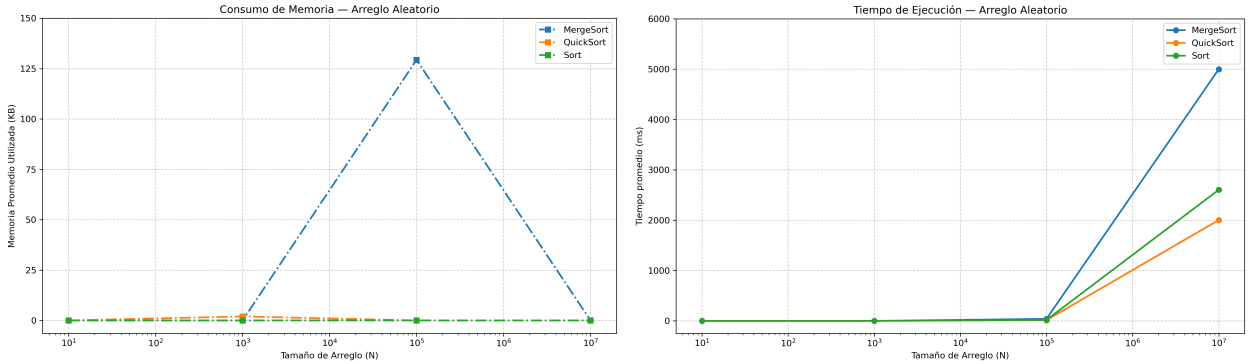

Figura 1: Comparación del tiempo de ejecución y consumo de memoria de arreglos aleatorios por cada algoritmo

Para este caso observamos que en memoria pasa un caso extraño, puesto a que los arreglos de 100000 presentaron el mayor consumo de memoria en MergeSort en vez de que los arreglos de mayor tamaño sean quienes lo presenten, es probable que se deba a la gestion de memoria del sistema operativo según lo investigado (Ver bibliografia)

En el caso de Quicksort presenciamos un fenomeno similar, pero a menor escala y en arreglos de 10000, mientras que en sort observamos que no presenta un consumo de memoria para nada notorio por la forma en el que este algoritmo opera.

En cuanto a tiempos, todos los algoritmos resultaron ser rapidos en arreglos de hasta 100000, pero cuando llegamos al tamaño maximo en el dataset estos escalan notoriamente, siendo mergesort el más tardio de los 3 en promedio de todos los casos, y con una diferencia notoria.

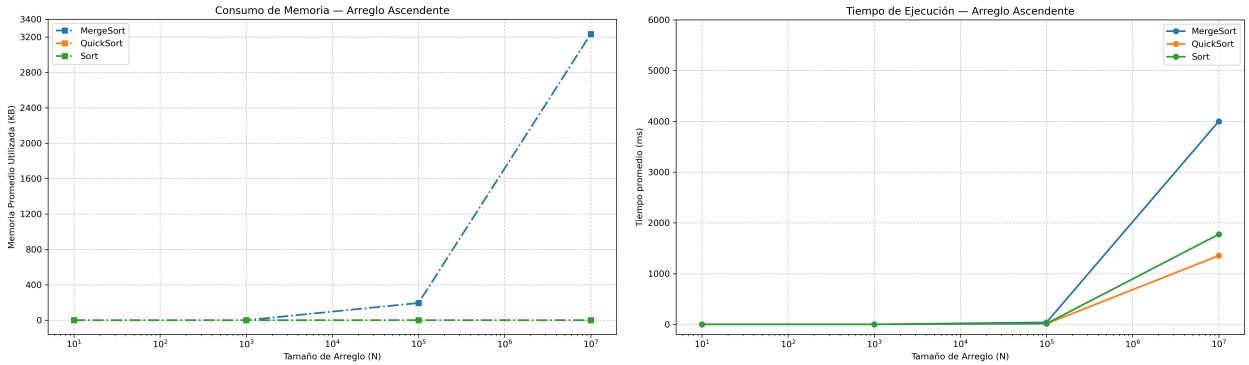

Figura 2: Comparación del tiempo de ejecución y consumo de memoria de arreglos ascendentes por cada algoritmo

En lo que respecta al consumo de memoria observamos un escenario más çomún", pues como se espera los arreglos de tamaño maximo presentan el mayor consumo de memoria para el caso del algoritmo MergeSort, es apreciable como el consumo de memoria se dispara entre los arreglos de 100000 y 10000000

En el caso de los otros algoritmos, estos mantienen un consumo bajo de memoria dado el enfoque de los mismos, por lo que se esperaba que MergeSort fuera el que "destacará.en este aspecto.

En cuanto al tiempo de ejecucion, el escenario es muy parecido al caso anterior con la diferencia de que en todos los algoritmos el tiempo es menor que en el caso aleatorio, por lo que se puede inferir que los algoritmos son más .eficientesçuando los arreglos vienen desordenados de esa forma.

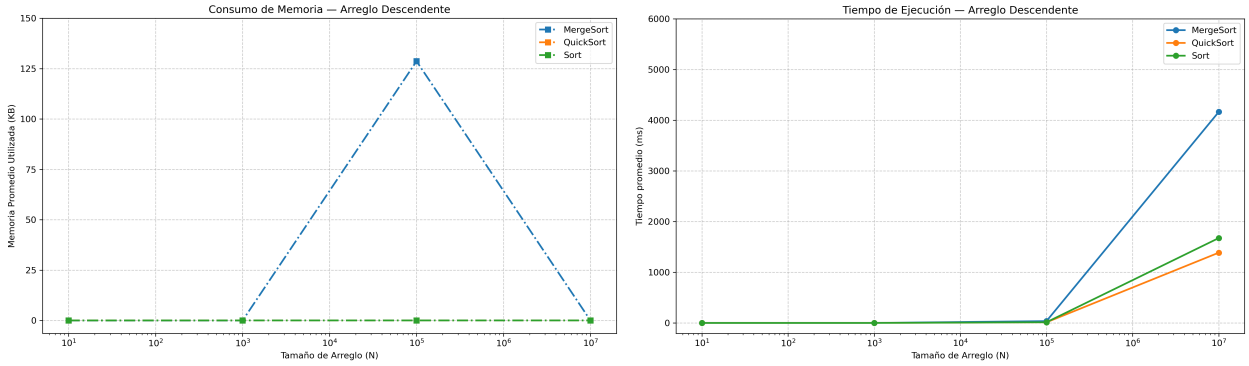

Figura 3: Comparación del tiempo de ejecución y consumo de memoria de arreglos descendentes por cada algoritmo

En el consumo de memoria tenemos un escenario parecido que en el caso de los arreglos aleatorios, pues MergeSort poseé tambien un mayor consumo de memoria en arreglos de 100000 y no en los de maximo tamaño, probablemte al motivo explicado anteriormente. Por otra parte, QuickSort y Sort no presentam un consumo notorio en este caso.

En los tiempos observamos un escenario parecido que en los arreglos ascendentes, tiempos menores que con los arreglos aleatorios, por lo que una vez más esta reduccion pueda deberse a la forma en el que los arreglos estaban desordenados.

Pasando a los algoritmos de multiplicacion de matrices, obtuvimos los siguientes resultados:

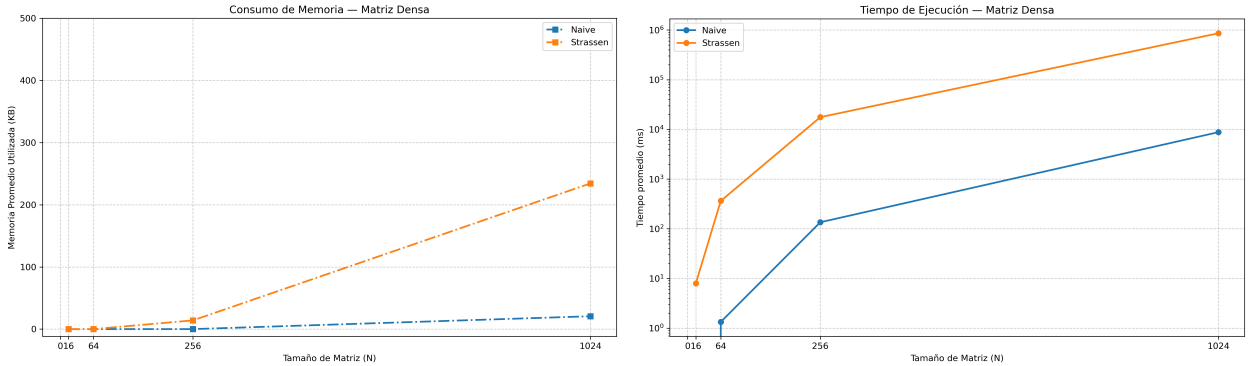

Figura 4: Comparación del tiempo de ejecución y consumo de memoria de matrices densas por cada algoritmo

Como podemos observar en priumera instancia en las matrices densas, el consumo de memoria de straseen es mayor independiente del tamaño de la matriz, esto debido a su enfoque de llamadas recursivas el cual aumenta el consumo de esta misma de forma significativa

En cambio naive, al ser un algoritmo de bucles y no recurrir a llamadas recursivas hace que su consumo de memoria sea muy inferior (Al menos para los tamaños tratados en este experimento)

En cuanto a tiempos de ejecucion, nos encontramos con que naive tambien presenta menores tiempos que strassen, incluso en las matrices de 1024x1024, esto se debe a que Strssen es generalmente mejor cuando se trata de matrices muy grandes, de ahí que naive sea más .ªpto"para los tamaños tratados aquí.

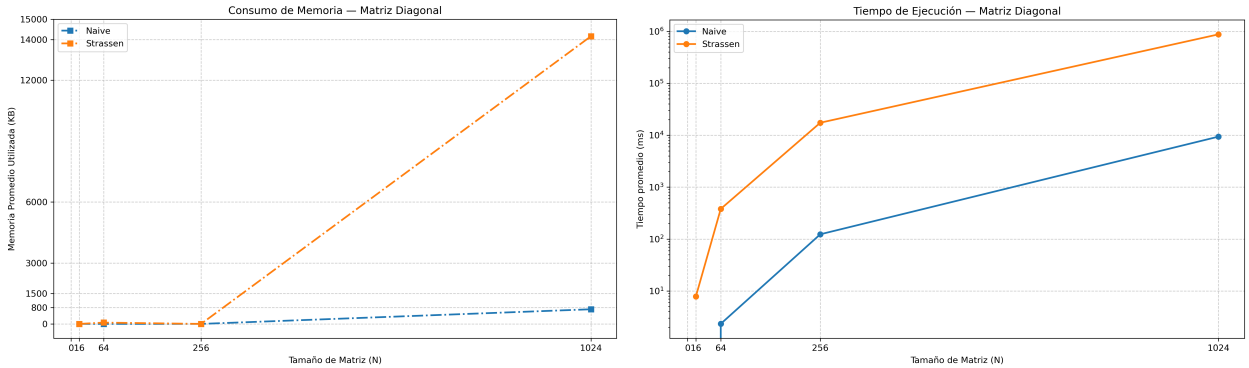

Figura 5: Comparación del tiempo de ejecución y consumo de memoria de matrices diagonales por cada algoritmo

Podemos notar que la tendencia se mantiene, strassen en matrices diagonales sigue presentando mayores tiempos de ejecucion y memoria que Naive, sin mencionar que acá fue donde Strassen presentó el mayor consumo de memoria promedio probablemente por el enfoque de las matrices en cuestion, aunque los tiempos promedio de ambos algoritmos por tamaño se mantienen similares a los de las matrices densas.

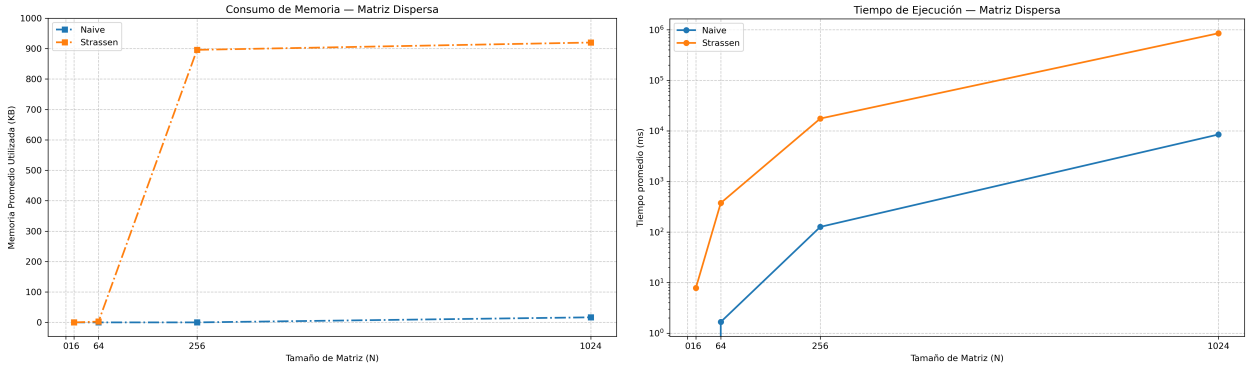

Figura 6: Comparación del tiempo de ejecución y consumo de memoria de matrices dispersas por cada algoritmo

Acá se aprecia que el consumo de memoria en strassen se dispara considerablemente apartir de las matrices de 256x256, muy probablemente al .ºrden"de estas mismas y que en promedio son capaces de ocupar más espacio de memoria de lo usual a partir de un tamaño más pequeño. Naive mantiene la tendencia de bajo consumo de memoria.

Y en cuanto a los tiempos de ejecucion, se mantienen con un promedio similar por tamaño según los graficos y comparado con los otros tipos de matrices.

3. Conclusiones

En conclusion, para los arreglos es posible concluir que Mergesort es al algoritmo más "lento"de los tres además de presentar el mayor consumo de memoria (Al menos en el escenario de este experimento). Mientras que Quicksort resulta ser el más rapido y eficiente en consumo de memoria independiente del tipo de arreglo. Para las matrices se concluye que Strassen es el algoritmo más "pesadoz que los tamaños de matrices con los que se trataron en este experimento pueden no ser los ideales para este. Mientras que Naive resultó ser el más rapido y eficiente para este escenario. Finalmente, los tipos de arreglos y matrices presentan mś impacto en el consumo de memoria que en el tiempo empleado independiente del algoritmo.